

3. Memory Management

1. Introduction of memory management
2. Basic memory management mechanism
3. Memory allocation
4. Swapping and paging
5. Virtual memory
6. Page replacement algorithm
7. Segmentation with paging

3. Memory Management

1. Introduction Of Memory Management

In a uni-programming system, main memory is divided into two parts: one part for the operating system (resident monitor, kernel) and one part for the user program currently being executed.

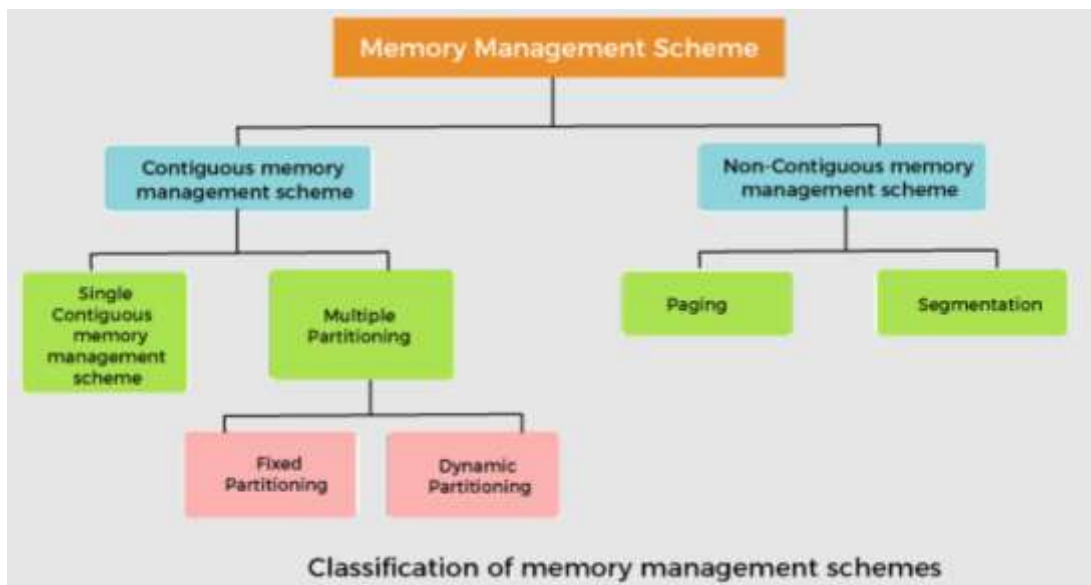
In a multiprogramming system, the “user” part of memory must be further subdivided to accommodate multiple processes. The task of subdivision is carried out dynamically by the operating system and is known as **memory management**.



2. Basic Memory Management Mechanism

- Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution.
- Memory management keeps track of each and every memory location, regardless of whether it is allocated to some process or it is free.
- It checks how much memory is to be allocated to processes.
- It decides which process will get memory at what time.
- It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

There are two Memory Management Techniques: **Contiguous**, and **Non-Contiguous**.



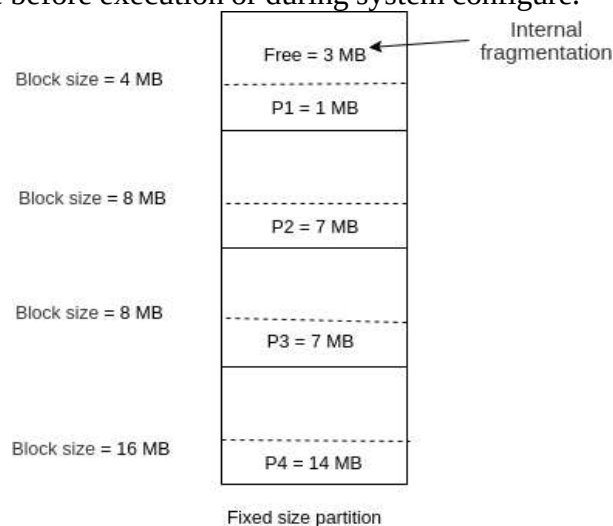
- In **Contiguous Technique**, executing process must be loaded entirely in main- memory. In contiguous memory allocation each process is contained in a single contiguous block of memory. Memory is divided into several fixed size partitions.
- Each partition contains exactly one process.

Contiguous Technique can be divided into:

1. Fixed (or static) partitioning
2. Variable (or dynamic) partitioning

Fixed Partitioning:

- This is the oldest and simplest technique used to put more than one processes in the main memory.
- In this partitioning, number of partitions (non-overlapping) in RAM are fixed but size of each partition may or may not be same.
- As it is contiguous allocation, hence no spanning is allowed.
- Here partitions are made before execution or during system configure.



Advantages of Fixed Partitioning –

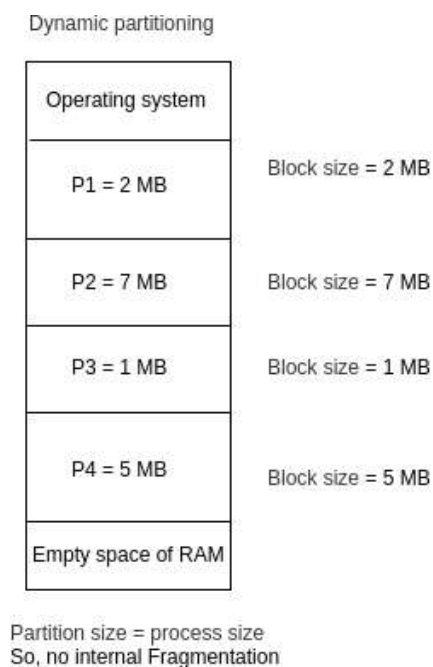
1. Easy to implement
2. Little OS overhead

Disadvantages of Fixed Partitioning –

- Internal Fragmentation
- External Fragmentation
- Limit process size
- Limitation on Degree of Multiprogramming

Variable Partitioning

- It is a part of Contiguous allocation technique.
- It is used to alleviate(suffering) the problem faced by Fixed Partitioning.
- In contrast with fixed partitioning, partitions are not made before the execution or during system configure.
- Various **features** associated with variable Partitioning-
 1. Initially RAM is empty and partitions are made during the run-time according to process's need instead of partitioning during system configure.
 2. The size of partition will be equal to incoming process.
 3. The partition size varies according to the need of the process so that the internal fragmentation can be avoided to ensure efficient utilisation of RAM.
 4. Number of partitions in RAM is not fixed and depends on the number of incoming process and Main Memory's size.

**Advantages of Variable Partitioning**

1. No Internal Fragmentation
2. No restriction on Degree of Multiprogramming
3. No Limitation on the size of the process

Disadvantages of Variable Partitioning

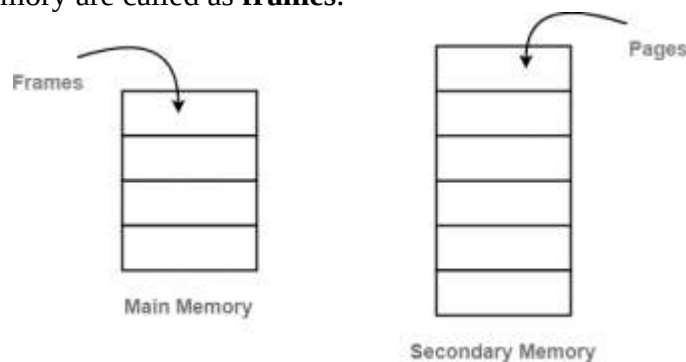
1. Difficult Implementation
2. External Fragmentation

Non-Contiguous Memory Management

In a **Non-Contiguous Memory Management** scheme, the program is divided into different blocks and loaded at different portions of the memory that need not necessarily be adjacent to one another. This scheme can be classified depending upon the size of blocks and whether the blocks reside in the main memory or not.

a. Paging

- Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory.
- This scheme permits the physical address space of a process to be non – contiguous.
- Paging is a fixed size partitioning scheme.
- In paging, secondary memory and main memory are divided into equal fixed size partitions.
- The partitions of secondary memory are called as **pages**.
- The partitions of main memory are called as **frames**.



- Each process is divided into parts where size of each part is same as page size.
- The size of the last part may be less than the page size.
- The pages of process are stored in the frames of main memory depending upon their availability.

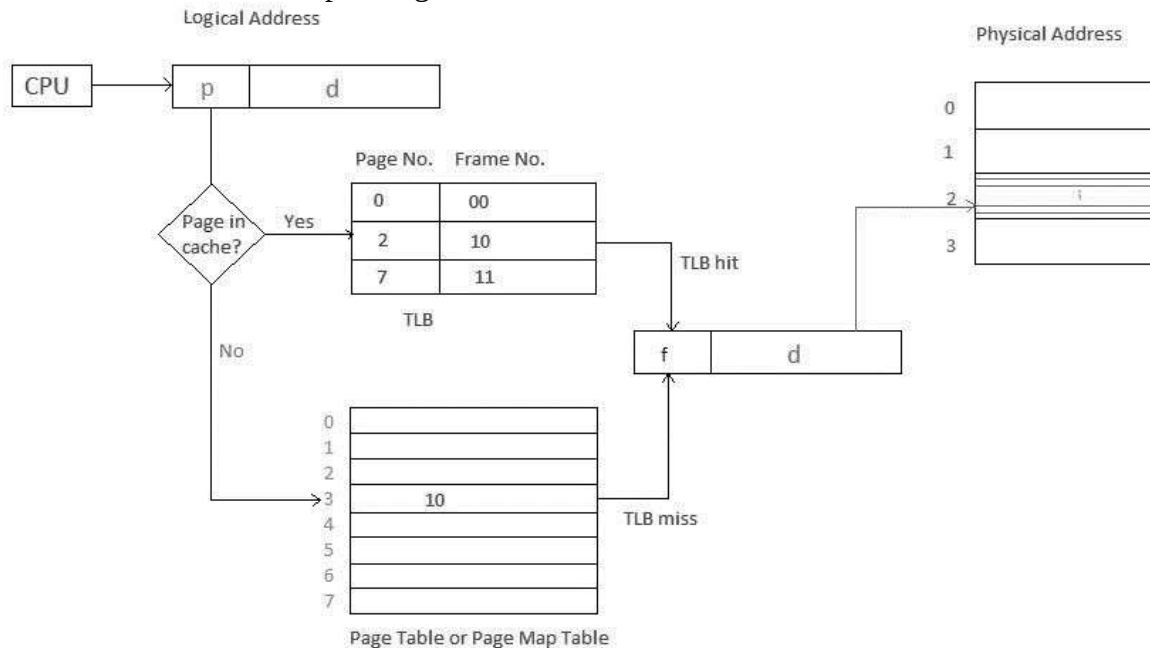
Address generated by CPU is divided into

- **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page offset(d):** Number of bits required to represent particular word in a page or page size of Logical Address Space or word number of a page or page offset.

Physical Address is divided into

- **Frame number(f):** Number of bits required to represent the frame of Physical Address Space or Frame number.
- **Frame offset(d):** Number of bits required to represent particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.
- The hardware implementation of page table can be done by using dedicated registers.

- But the usage of register for the page table is satisfactory only if page table is small.
- If page table contain large number of entries then we can use TLB (translation Look-aside buffer), a special, small, fast look up hardware cache.
 - The TLB is associative, high speed memory.
 - Each entry in TLB consists of two parts: a tag and a value.
 - When this memory is used, then an item is compared with all tags simultaneously. If the item is found, then corresponding value is returned.



Advantages-

The advantages of paging are-

- It allows to store parts of a single process in a non-contiguous fashion.
- It solves the problem of external fragmentation.

Disadvantages-

The disadvantages of paging are-

- It suffers from internal fragmentation.
- There is an overhead of maintaining a page table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.

Page Table-

- Page table is a data structure.
- It maps the page number referenced by the CPU to the frame number where that page is stored.

Characteristics-

- Page table is stored in the main memory.
- Number of entries in a page table = Number of pages in which the process is divided.
- Page Table Base Register (PTBR) contains the base address of page table.
- Each process has its own independent page table.
- Page Table Base Register (PTBR) provides the base address of the page table.
- The base address of the page table is added with the page number referenced by the CPU.

- It gives the entry of the page table containing the frame number where the referenced page is stored.

Page Table Entry format:

- A page table entry contains several information about the page.
- The information contained in the page table entry varies from operating system to operating system.
- The most important information in a page table entry is frame number, Present / Absent Bit, Protection Bit, Reference Bit, Dirty Bit (Modified bit)

Page Replacement Algorithm

In an operating system that uses paging for memory management, a page replacement algorithm is needed to decide which page needs to be replaced when new page comes in.

Principle of Page Replacement

The page replacement is done by swapping. The required pages from backup storage to main memory and vice-versa. This swapping is done by checking the contents of physical memory. If there is a free frame in the memory then swap-in the required page into the frame which is free.

In case, if there is no frame in physical memory then first find the frame which is not currently in use. The content of this frame is swapped-out from memory to backup storage. Then bring the required page in the frame which is now free.

As studied in *Demand Paging*, only certain pages of a process are loaded initially into the memory. This allows us to get more number of processes into the memory at the same time. but what happens when a process requests for more pages and no free memory is available to bring them in. Following steps can be taken to deal with this problem:

1. Put the process in the wait queue, until any other process finishes its execution thereby freeing frames.
2. Or, remove some other process completely from the memory to free frames.
3. Or, find some pages that are not being used right now, move them to the disk to get free Frames.

Page Fault – A page fault happens when a running program accesses a memory page that is mapped into the virtual address space, but not loaded in physical memory.

This technique is called **Page Replacement** and is most commonly used.

Page Replacement Algorithm

There are many different page replacement algorithms. We evaluate an algorithm by running it on a particular *string of memory reference* and computing the *number of page faults*. The string of memory references is called **reference string**. Reference strings are generated artificially or by tracing a given system and recording the address of each memory reference.

To determine the number of page faults for a particular reference string and page replacement algorithm, we also need to know the number of page frames available. *As the number of frames available increase, the number of page faults will decrease.*

There are three types of Page Replacement Algorithms. They are:

- First In First Out Page Replacement Algorithm
- Optimal Page Replacement Algorithm
- Least Recently Used (LRU) Page Replacement Algorithm

FIFO (First In First Out) Page Replacement

- Simplest page replacement algorithm
- Oldest page in main memory is the one which will be selected for replacement. Easy to implement, keep a list, replace pages from the tail and add new pages.

Advantages: Easy to understand and program. Distributes fair chance to all.

Problems: FIFO is likely to replace heavily (or constantly) used pages and they are still needed for further processing.

Q1.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
			1																

page frames

No. of pagefault=15

Points to Remember

Page Not Found - - - > Page Fault

Page Found - - - > Page Hit

Q2.

Reference String:

5, 0, 1, 0, 2, 3, 0, 2, 4, 3, 3, 2, 0, 2, 1, 2, 7, 0, 1, 1, 0

		1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1
	0	0	0	0	3	3	3	3	3	3	2	2	2	2	2	2	0	0	0	0
5	5	5	5	2	2	2	2	4	4	4	4	4	4	4	4	7	7	7	7	7
			H				H		H	H		H	H		H			H	H	H
1	2	3	4	5	6	7	8	9	10	11										

Total Page Fault= 11

Number of Hit= 10

Rate of page fault = No. of page fault/No. of frames

$$11/3 = 3.6$$

Q2.

Reference String:

6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0

S. no	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
F3				2	2	2	4	4	4	2	2	2	2	2	2	2	2	2	2	0
F2		1	1	1	1	3	3	3	0	0	0	0	0	0	0	3	3	3	3	3
F1	6	6	6	6	0	0	0	6	6	6	1	1	1	1	1	1	1	1	1	1
Hit (H)/ Fault (F)	F	F	H	F	F	F	F	F	F	F	F	H	H	H	H	F	H	H	H	F

Number of Page Hits = 8

Number of Page Faults = 12

he Ratio of Page Hit to the Page Fault = 8 : 12 - - - > 2 : 3 - - - > 0.66

The Page Hit Percentage = $8 * 100 / 20 = 40\%$

The Page Fault Percentage = $100 - \text{Page Hit Percentage} = 100 - 40 = 60\%$

Belady's Anomaly:

Note: For some page-replacement algorithms, the page fault rate may increase as the number of allocated frames increases. This most unexpected result is known as **Belady's anomaly**.

For example if we consider the following page reference string

0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4 with no of frame 3, we get total 9 page fault as shown in fig (a).

But if we increase the no. of frame from 3 to 4 we get 10 page fault as shown in fig (b)

Case 1: Frame=3

		2	2	2	1	1	1	1	1	3	3
	1	1	1	0	0	0	0	0	2	2	2
0	0	0	3	3	3	4	4	4	4	4	4
							H	H			H

Fig (a)

Page Fault= 9

Case 2: Frame=4

			3	3	3	3	3	3	2	2	2
		2	2	2	2	2	2	1	1	1	1
	1	1	1	1	1	1	0	0	0	0	4
0	0	0	0	0	0	4	4	4	4	3	3
				H	H						

Fig (b)

Page Fault= 10

Advantages: ➤ It is simple and easy to understand & implement.

Disadvantages:

➤ The process effectiveness is low.

➤ When we increase the number of frames while using FIFO, we are giving more memory to processes. So, page fault should decrease, but here the page faults are increasing. This problem is called as **Belady's anomaly**.

LRU (Least-Recently-Used) Page Replacement

In this algorithm, the page that has been not used for longest period of time is selected for replacement. Example: Reference string is:

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1			
	0	0	0		0		0	0	3	3		3		0		0			
		1	1		3		3	2	2	2		2		2		7			

page frames

5, 0, 1, 0, 2, 3, 0, 2, 4, 3, 3, 2, 0, 2, 1, 2, 7, 0, 1, 1, 0

		<u>1</u>	1	1	<u>3</u>	3	3	<u>4</u>	4	4	4	<u>0</u>	0	0	0	<u>7</u>	7	7	7	7
	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	0	0	<u>3</u>	<u>3</u>	3	3	3	<u>1</u>	1	1	<u>0</u>	0	0	0
<u>5</u>	5	5	5	<u>2</u>	2	<u>2</u>	<u>2</u>	2	2	2	<u>2</u>	2	<u>2</u>	2	<u>2</u>	2	2	<u>1</u>	<u>1</u>	1
			H			H	H			H	H		H		H				H	H

Total Page Fault= 12

Number of Hit= 9

Rate of page fault= No. of page fault/No. of frames = 12/3= 4

6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 2, 0

S. no	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
F3				2	2	2	4	4	4	2	2	2	2	2	2	2	2	2	2	2
F2		1	1	1	1	3	3	3	0	0	0	0	0	0	0	0	0	1	1	1
F1	6	6	6	6	0	0	0	6	6	6	1	1	1	1	1	3	3	3	3	0
Hit (H)/ Fault (F)	F	F	H	F	F	F	F	F	F	F	F	H	H	H	H	F	H	F	H	F

Number of Page Hits = 7

Number of Page Faults = 13

The Ratio of Page Hit to the Page Fault = 7 : 12 - - - > 0.5833 : 1

The Page Hit Percentage = 7 * 100 / 20 = 35%

The Page Fault Percentage = 100 - Page Hit Percentage = 100 - 35 = 65%

Advantages:

- It is open for full analysis.
- In this, we replace the page which is least recently used, thus free from Belady's Anomaly.
- Easy to choose page which has faulted and hasn't been used for a long time.

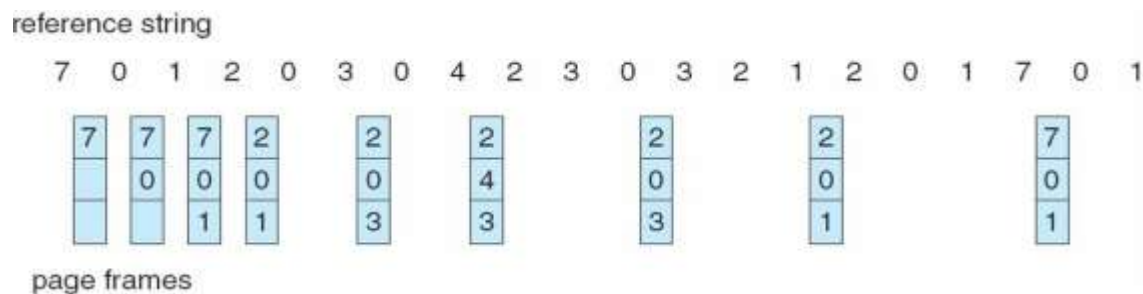
Disadvantages:

- It requires additional Data Structure to be implemented.
- Hardware assistance is high.

Optimal Page Replacement

This algorithm state that: "replace that page which will not be used for longest period of time" i.e. future knowledge of reference string is required.

One result of the discovery of Belady's anomaly was the search for an optimal page replacement algorithm. An optimal page-replacement algorithm has the lowest page-fault rate of all algorithms, and will never suffer from Belady's anomaly. Such an algorithm does exist, and has been called OPT or MIN.



Example: Reference string is

5, 0, 1, 0, 2, 3, 0, 2, 4, 3, 3, 2, 0, 2, 1, 2, 7, 0, 1, 1, 0

		1	1	1	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
	0	0	0	0	0	0	0	4	4	4	4	0	0	0	0	0	0	0	0	0
5	5	5	5	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7	7
			H			H	H		H	H	H		H		H		H	H	H	H

Total Page Fault= 9

Number of Hit= 12

Rate of page fault= No. of page fault/No. of frames = 9/3= 3

Reference String: 6, 1, 1, 2, 0, 3, 4, 6, 0, 2, 1, 2, 1, 2, 0, 3, 2, 1, 4, 0

S. no	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
F3				2	2	2	4	4	4	4	4	4	4	4	4	3	3	3	4	4
F2		1	1	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1
F1	6	6	6	6	0	0	0	6	6	2	2	2	2	2	2	2	2	2	2	0
Hit (H)/ Fault (F)	F	F	H	F	F	F	F	H	H	F	F	H	H	H	H	F	H	F	F	F

Number of Page Hits = 8

Number of Page Faults = 12

The Ratio of Page Hit to the Page Fault = $8 : 12 \rightarrow 2 : 3 \rightarrow 0.66$

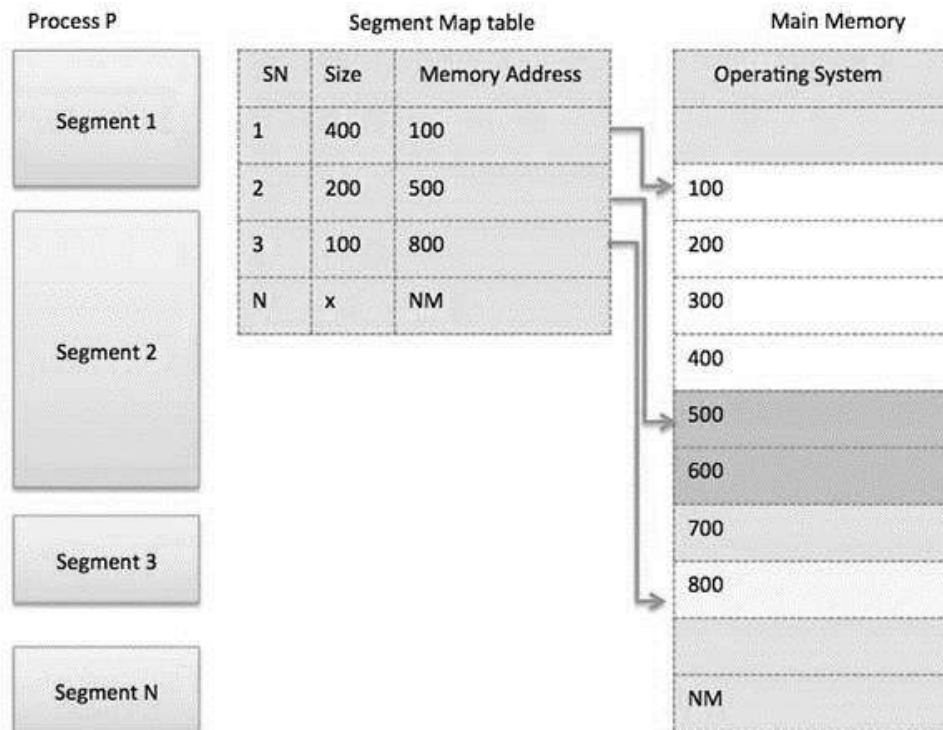
The Page Hit Percentage = $8 * 100 / 20 = 40\%$

The Page Fault Percentage = $100 - \text{Page Hit Percentage} = 100 - 40 = 60\%$

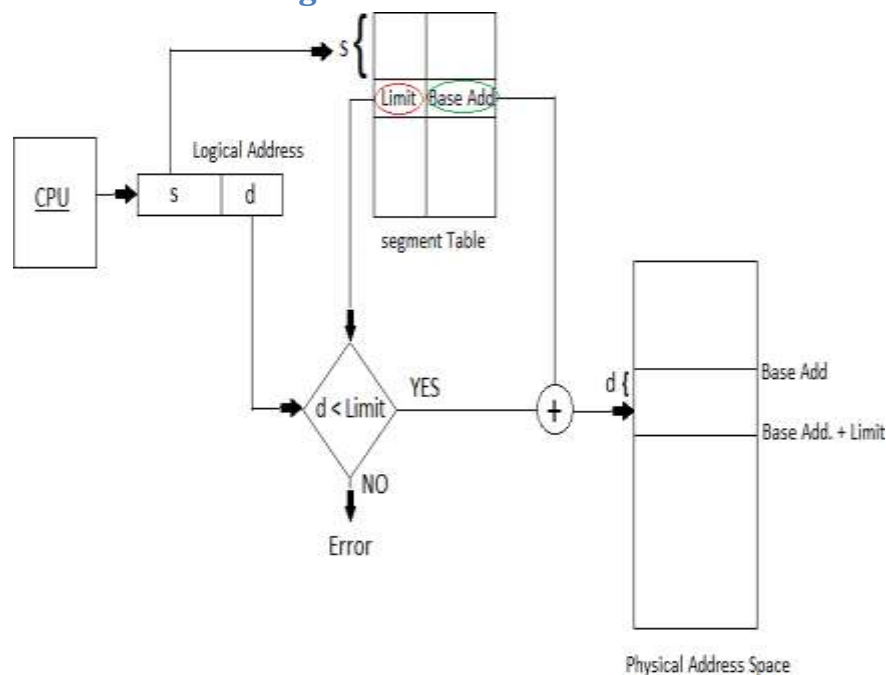
Segmentation with paging

b. Segmentation

- Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions.
- Each segment is actually a different logical address space of the program.
- When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.
- Segmentation memory management works very similar to paging but here segments are of variable-length where as in paging pages are of fixed size.
- The operating system maintains a segment map table for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory.
- For each segment, the table stores the starting address of the segment and the length of the segment.
- A reference to a memory location includes a value that identifies a segment and an offset.
- **Segment Table** – It maps two-dimensional Logical address into one- dimensional Physical address. It's each table entry has:
 1. **Base Address:** It contains the starting physical address where the segments reside in memory.
 2. **Limit:** It specifies the length of the segment.



Translation of Two Dimensional Logical Address to one dimensional Physical Address.



Address generated by the CPU is divided into:

- **Segment number (s):** Number of bits required to represent the segment.
- **Segment offset (d):** Number of bits required to represent the size of the segment.

Advantages of Segmentation –

- No Internal fragmentation.
- Segment Table consumes less space in comparison to Page table in paging.

Disadvantage of Segmentation –

- As processes are loaded and removed from the memory, the free memory space is broken into little

pieces, causing External fragmentation.

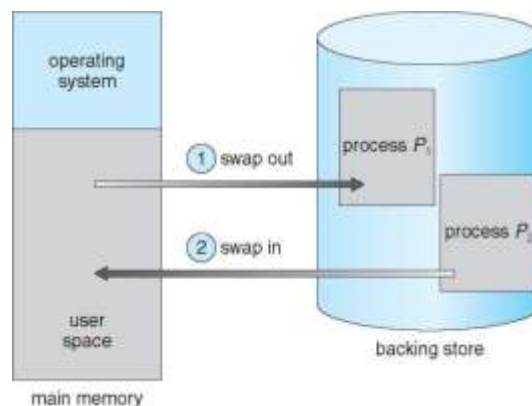
Swapping

In multiprogramming environment, sometimes there is no enough memory to hold all processes that is being executed.

How to handle this problem?—swapping.

A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution both the memory and CPU.

In multiprogramming processes will be swapped in and out manytimes before its completion. In some swapping systems, one processes occupies the main memory at once; when process can no longer to continue it yield (provide).



Virtual Memory

Virtual Memory is a space where large programs can store themselves in form of pages while their execution and only the required pages or portions of processes are loaded into the main memory. This technique is useful as large virtual memory is provided for user programs when a very small physical memory is there.

In real scenarios, most processes never need all their pages at once, for following reasons :

- Error handling code is not needed unless that specific error occurs, some of which are quite rare.
- Arrays are often over-sized for worst-case scenarios, and only a small fraction of the arrays are actually used in practice.
- Certain features of certain programs are rarely used.

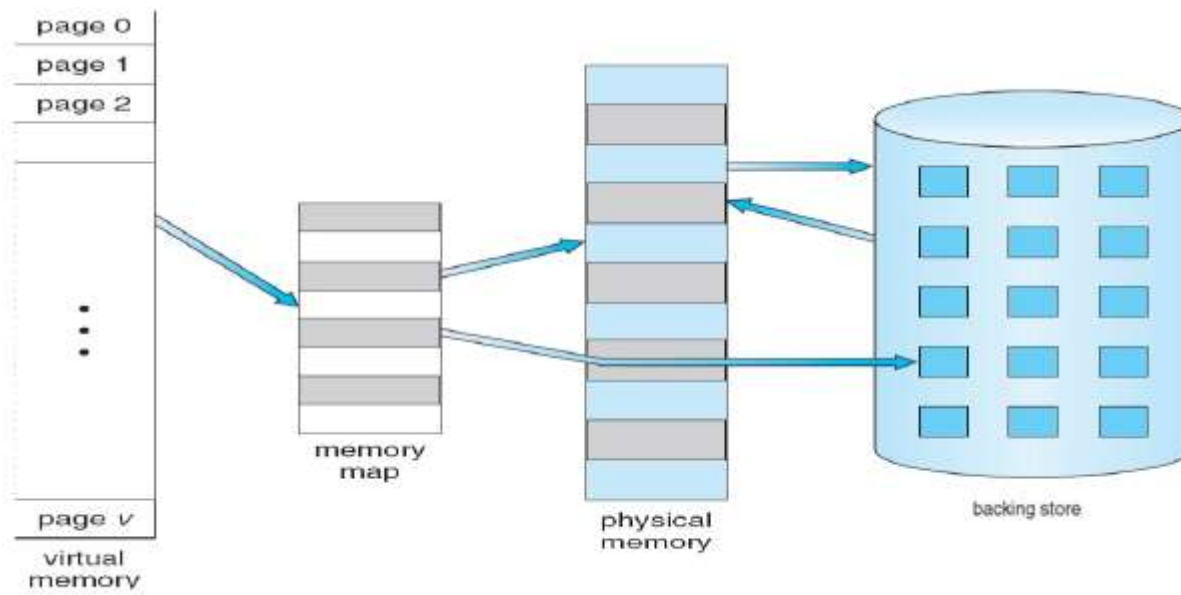


Fig. Diagram showing virtual memory that is larger than physical memory.

Virtual memory is commonly implemented by demand paging. It can also be implemented in a segmentation system. Demand segmentation can also be used to provide virtual memory.

Benefits of having Virtual Memory :

1. Large programs can be written, as virtual space available is huge compared to physical memory.
2. Less I/O required, leads to faster and easy swapping of processes.
3. More physical memory available, as programs are stored on virtual memory, so they occupy very less space on actual physical memory.

Memory allocation

There are two methods namely, multiple partition method and a general fixed partition method.

In multiple partition method, the memory is divided into several fixed size partitions. One process occupies each partition. This scheme is rarely used nowadays.

Degree of multiprogramming depends on the number of partitions. Degree of multiprogramming is the number of programs that are in the main memory. The CPU is never left idle in multiprogramming. This was used by IBM OS/360 called **Multiprogramming with a Fixed number of Tasks MFT**.

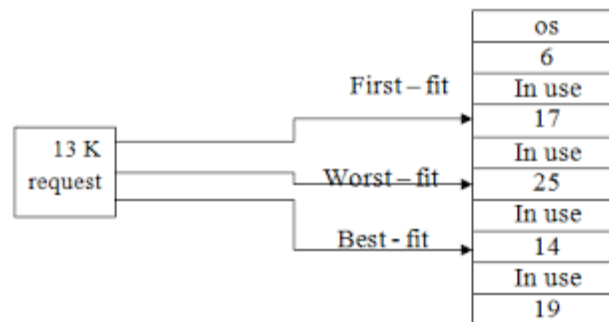
Generalization of fixed partition scheme is used in MVT.

MVT stands for **Multiprogramming with a Variable number of Tasks**. The Operating System keeps track of which parts of memory are available and which is occupied. This is done with the help of a table that is maintained by the Operating System. Initially the whole available memory is treated as one large block of memory called a **hole**.

Now there comes a general dynamic storage allocation problem. The following are the solutions to the dynamic storage allocation problem.

1. **First fit:** The first hole that is large enough is allocated. Searching for the holes starts from the beginning of the set of holes or from where the previous first fit search ended.
2. **Best fit:** The smallest hole that is big enough to accommodate the incoming process is allocated. If the available holes are ordered, then the searching can be reduced.
3. **Worst fit:** The largest of the available holes is allocated.

Example



First and best fits decrease time and storage utilization. First fit is generally faster.

Problem:

Given memory partitions of 100K, 500K, 200K, 300K, and 600K (in order), how would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)? Which algorithm makes the most efficient use of memory?

Answer:

- a. First-fit:
 - b. 212K is put in 500K partition
 - c. 417K is put in 600K partition
 - d. 112K is put in 288K partition (new partition 288K = 500K - 212K)
 - e. 426K must wait
- f. Best-fit:
 - g. 212K is put in 300K partition
 - h. 417K is put in 500K partition
 - i. 112K is put in 200K partition
 - j. 426K is put in 600K partition
- k. Worst-fit:
 - l. 212K is put in 600K partition
 - m. 417K is put in 500K partition
 - n. 112K is put in 388K partition
 - o. 426K must wait

In this example, Best-fit turns out to be the best.

Assignment

- What is Main Memory?
- What is Memory Management?
- Why Memory Management is Required?
- Logical Address Space and Physical Address Space
- Static and Dynamic Loading
- Static and Dynamic Linking
- Swapping
- Contiguous Memory Allocation
 - Memory Allocation
 1. First Fit
 2. Best Fit
 3. Worst Fit
 - Fragmentation
 0. Internal Fragmentation
 1. External Fragmentation
 - Paging

What is Memory Management?

In a multiprogramming computer, the Operating System resides in a part of memory, and the rest is used by multiple processes. The task of subdividing the memory among different processes is called Memory Management. Memory management is a method in the operating system to manage operations between main memory and disk during process execution. The main aim of memory management is to achieve efficient utilization of memory.

Why Memory Management is Required?

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Logical and Physical Address Space

- **Logical Address Space:** An address generated by the CPU is known as a “Logical Address”. It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.
- **Physical Address Space:** An address seen by the memory unit (i.e the one loaded into the memory address register of the memory) is commonly known as a “Physical Address”. A Physical address is also known as a Real address. The set of all physical addresses corresponding to these logical addresses is known as Physical address space. A physical address is computed by MMU. The run-time mapping from

virtual to physical addresses is done by a hardware device Memory Management Unit(MMU). The physical address always remains constant.

Static and Dynamic Loading

Loading a process into the main memory is done by a loader. There are two different types of loading :

- **Static Loading:** Static Loading is basically loading the entire program into a fixed address. It requires more memory space.
- **Dynamic Loading:** The entire program and all data of a process must be in physical memory for the process to execute. So, the size of a process is limited to the size of physical memory. To gain proper memory utilization, dynamic loading is used. In dynamic loading, a routine is not loaded until it is called. All routines are residing on disk in a relocatable load format. One of the advantages of dynamic loading is that the unused routine is never loaded. This loading is useful when a large amount of code is needed to handle it efficiently.

Static and Dynamic Linking

To perform a linking task a linker is used. A linker is a program that takes one or more object files generated by a compiler and combines them into a single executable file.

- **Static Linking:** In static linking, the linker combines all necessary program modules into a single executable program. So there is no runtime dependency. Some operating systems support only static linking, in which system language libraries are treated like any other object module.
- **Dynamic Linking:** The basic concept of dynamic linking is similar to dynamic loading. In dynamic linking, “Stub” is included for each appropriate library routine reference. A stub is a small piece of code. When the stub is executed, it checks whether the needed routine is already in memory or not. If not available then the program loads the routine into memory.

What is Demand Paging?

Demand paging can be described as a memory management technique that is used in operating systems to improve memory usage and system performance. Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU.

In demand paging, the operating system loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start. A page fault occurred when the program needed to access a page that is not currently in memory. The operating system then loads the required pages from the disk into memory and updates the page tables accordingly. This process is transparent to the running program and it continues to run as if the page had always been in memory.

<https://www.geeksforgeeks.org/memory-management-in-operating-system/>

<https://www.vbspu.ac.in/e-content/Page-Replacement-Algorithms.pdf>

<https://www.javatpoint.com/os-page-replacement-algorithms>

<https://www.javatpoint.com/os-page-replacement-algorithms>

<https://aissmospoly.org.in/wp-content/uploads/2020/01/Memory-management-.pdf>